

Kube-Proxy와 Service

Kubernetes에서 Pod는 각각 고유한 IP를 가지고 서로 직접 통신할 수 있습니다. 하지만 실제 애플리케이션이 특정 Pod의 IP를 직접 호출하는 방식은 문제가 있습니다.

```
frontend Pod → 10.244.2.20
```

Pod는 Deployment의 배포, 장애, 스케일 아웃·인에 따라 언제든지 삭제되고 새로 만들어질 수 있습니다. 새 Pod는 대개 다른 IP를 받습니다.

```
기존 API Pod: 10.244.2.20
```

```
새 API Pod: 10.244.2.31
```

따라서 frontend가 10.244.2.20 을 코드나 설정에 직접 저장하고 있다면, API Pod가 교체되는 순간 통신이 깨집니다.

Kubernetes 공식 문서 중 Service [Service](#)에서는 Service를 아래와 같이 정의하고 있습니다.

The Service API, part of Kubernetes, is an abstraction to help you expose groups of Pods over a network. Each Service object defines a logical set of endpoints (usually these endpoints are Pods) along with a policy about how to make those pods accessible.

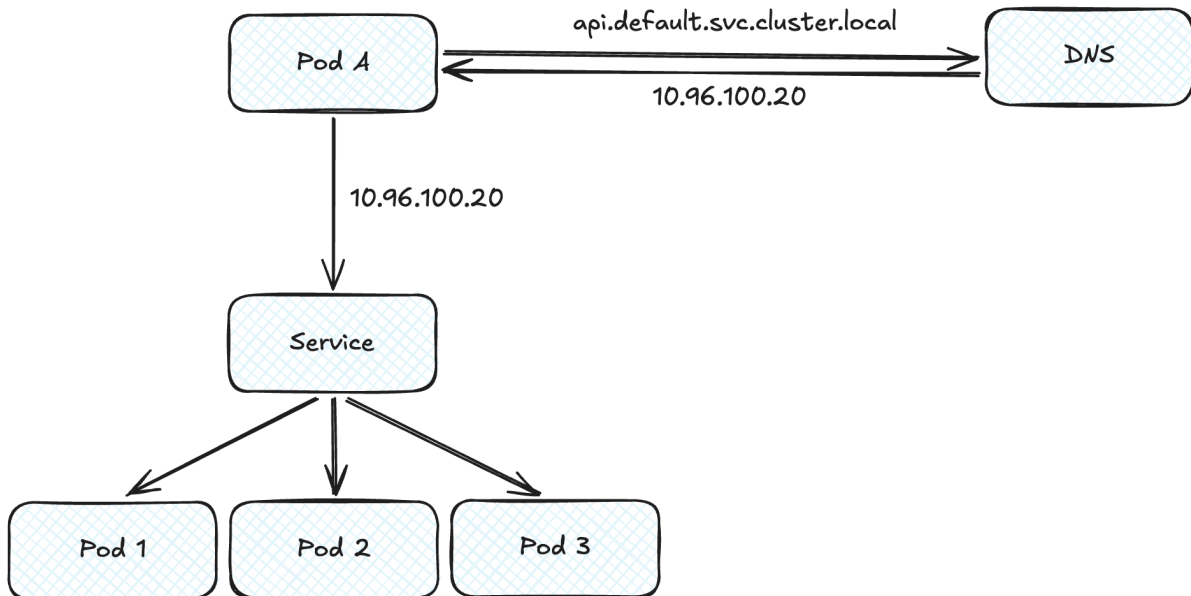
이 문서의 내용을 요약하면, Service는 다음과 같이 정의할 수 있습니다.

"Service는 하나 이상의 Pod에서 실행되는 네트워크 애플리케이션을 노출하는 방법이며, 실제 backend Pod가 변경되더라도 클라이언트가 그 변화를 직접 알 필요 없도록 한다."

1. Service

일반적인 Service는 다음 두 가지를 제공합니다.

- 고정된 DNS 이름
- 고정된 가상 IP(ClusterIP)



하지만 위 다이어그램은 제가 정확하게 그린 것이 아닙니다. 왜냐하면 Service는 요청을 전달하는 프로세스나 Pod가 아니기 때문입니다. 그렇다면 어떻게 Service가 Endpoint로 요청을 전송하는 건지 궁금해져서, 아래에서 그 부분에 대해 다뤄보겠습니다.

1.1. Service EndpointSlice

Service는 **label selector**를 사용해 backend Pod 집합을 선택합니다.

아래에서 간단한 `api` 라는 Service를 예시로 들어보겠습니다. `api` 서비스는 `app: api` 라는 Label을 가진 Pod를 Backend로 선택합니다.

```

apiVersion: v1
kind: Service
metadata:
  name: api
  namespace: default
spec:
  selector:
    app: api
  ports:
    - port: 80
      targetPort: 8080
  
```

현재 Kubernetes에 `api-1`, `api-2`, `api-3`, `frontend` 와 같이 여러 개의 Pod가 있다고 했을 때, `app:api` 라는 Label을 가진 Pod만이 `api` 서비스의 Backend가 됩니다.

```

Service: api
selector: app=api
  
```

```
Pod api-1: app=api → 선택됨
Pod api-2: app=api → 선택됨
Pod api-3: app=api → 선택됨
Pod frontend: app=frontend → 선택되지 않음
```

Service는 포트도 구분합니다. Service는 `port` 와 `targetPort` 두 가지가 있습니다.

- `port`: Service가 수신할 포트
- `targetPort`: 요청을 전달할 Backend의 포트

Service가 selector로 Pod를 선택하면 Kubernetes control plane은 **EndpointSlice**를 관리합니다. 그리고 EndpointSlice에는 Service의 실제 backend 주소가 기록됩니다. 간략한 예시는 아래와 같습니다.

```
Service: api
ClusterIP: 10.96.100.20:80

EndpointSlice:
- 10.244.1.11:8080    Ready
- 10.244.2.20:8080    Ready
- 10.244.2.21:8080    Ready
```

즉, Service는 “고정된 대표 주소”이고, EndpointSlice는 “현재 실제로 요청을 처리할 수 있는 Pod 목록”입니다.

Pod가 새로 생성되어 Ready 상태가 되면 EndpointSlice에 추가되고, Pod가 종료되거나 Ready 상태가 아니게 되면 backend 목록에서 제외됩니다.

Kubernetes EndpointSlice 공식 문서

In Kubernetes, an EndpointSlice contains references to a set of network endpoints. The control plane automatically creates EndpointSlices for any Kubernetes Service that has a `selector` specified. These EndpointSlices include references to all the Pods that match the Service selector. EndpointSlices group network endpoints together by unique combinations of IP family, protocol, port number, and Service name. The name of a EndpointSlice object must be a valid `DNS subdomain name`.

그런데 앞서 Service는 요청을 실제로 처리하는 프로세스 혹은 Pod가 아니라고 했는데, 그럼 어떻게 Service로부터 Pod에 요청이 도달하게 되는지 정리해보도록 하겠습니다.

2. Kube-Proxy

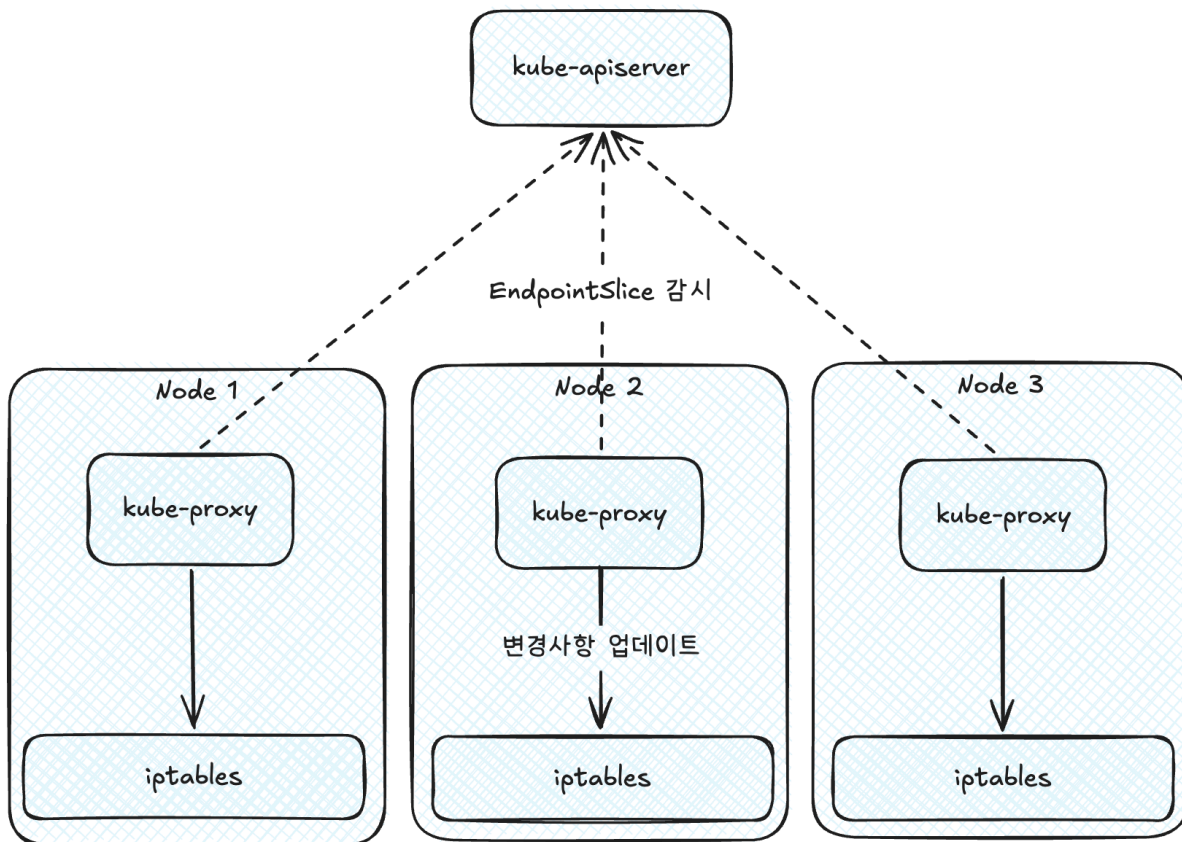
ClusterIP는 일반적으로 Service를 표현하는 가상 IP 입니다.

Service ClusterIP: 10.96.100.20

이 IP에 실제 애플리케이션 프로세스가 bind되어 요청을 받는 것은 아닙니다. 대신 각 Node의 kube-proxy가 Service와 EndpointSlice 정보를 감시하고, 해당 Service IP로 가는 패킷을 실제 backend Pod IP로 보내도록 Node의 데이터 플레인을 구성합니다.

Node의 데이터 플레인? 은 iptables 혹은 nftables의 규칙을 말합니다. (물론 CNI에 따라 달라질 수 있지만 기본값으로 보겠습니다)

다시 정리하면, Kube-Proxy가 모든 Service의 EndpointSlice에 변경사항이 있는지를 감시하고, 변경사항이 있으면 자기 노드의 iptables 규칙을 업데이트하는 것입니다.



따라서 Service는 요청을 컨트롤하는 실제 프로세스가 아니라, 각 노드의 라우팅 규칙을 업데이트하는 Configuration 정도로 이해하면 좋을 것 같습니다.

Pod → Linux kernel의 kube-proxy 규칙 → Pod

Kube-Proxy는 다음 모드를 지원합니다.

모드	역할
iptables	iptables/netfilter 규칙으로 Service 트래픽을 backend로 전달
ipvs	Linux IPVS의 가상 서버·로드밸런싱 기능 사용
nftables	nftables 규칙으로 Service 트래픽을 backend로 전달

iptables 와 nftables 모드에서는 기본적으로 Endpoint 중 하나를 무작위로 선택하는 규칙을 만들 수 있습니다. IPVS는 kernel의 IPVS 로드밸런싱 기능을 이용합니다.

이렇게 해서 여러 Backend (Endpoint) 중에서 하나로 자동으로 로드밸런싱이 됩니다.

2.1. Kube-Proxy DNAT?

다음과 같은 Service가 있다고 가정해보겠습니다.

```
Service: api
ClusterIP: 10.96.100.20:80

EndpointSlice:
  ① 10.244.1.11:8080
  ② 10.244.2.20:8080
  ③ 10.244.2.21:8080
```

클라이언트 Pod가 10.96.100.20:80 으로 패킷을 보내면, 해당 Node의 kube-proxy 규칙이 Endpoint 중 하나를 선택합니다.

```
10.96.100.20:80
|
├── ① 10.244.1.11:8080
├── ② 10.244.2.20:8080 ← 이번 연결에서 선택
└── ③ 10.244.2.21:8080
```

그럼 선택된 backend로 보내기 위해 목적지 주소와 포트가 변경됩니다.

```
변경 전
Destination: 10.96.100.20:80 (Service ClusterIP)
```

변경 후

Destination: 10.244.2.20:8080 (선택된 Pod)

이렇게 기존의 목적지(Service의 ClusterIP) 에서 새로운 목적지(Pod의 IP)로 목적지 IP를 변경하는 것을 **DNAT(Destination Network Address Translation)** 라고 합니다.

근데 Kubernetes의 Pod-to-Pod 통신은 NAT 없이 직접 통신한다고 했던 거 같아서 뭐지? 싶었습니다. 그런데 Kubernetes 공식문서 중 [Services, Load Balancing, and Networking](#) 에서는 다음과 같이 정의하고 있습니다.

- The *pod network* (also called a cluster network) handles communication between pods. It ensures that (barring intentional network segmentation):

- All pods can communicate with all other pods, whether they are on the same [node] (<https://kubernetes.io/docs/concepts/architecture/nodes/>) or on different nodes. Pods can communicate with each other directly, without the use of proxies or address translation (NAT).
- Agents on a node (such as system daemons, or kubelet) can communicate with all pods on that node.

- The [Service](#) API lets you provide a stable (long lived) IP address or hostname for a service implemented by one or more backend pods, where the individual pods making up the service can change over time.

- Kubernetes automatically manages [EndpointSlice] (<https://kubernetes.io/docs/concepts/services-networking/endpoint-slices/>) objects to provide information about the pods currently backing a Service.
- A service proxy implementation monitors the set of Service and EndpointSlice objects, and programs the data plane to route service traffic to its backends, by using operating system or cloud provider APIs to intercept or rewrite packets.

NAT를 사용하지 않고 직접 통신해야 하는 것은 Pod와 Pod 간의 통신이고, Service는 다른 개념입니다. Pod - Service ClusterIP 는 가상 Service IP를 실제 Pod IP로 바꾸기 위해 목적지 변환이 필요합니다.